

# Iterator e Chain of Responsibility

Dois padrões de comportamento: um para **percorrer** elementos, outro para **processar** solicitações.

# O que vamos ver

---

- 1. Onde eles ficam no catálogo**  
as três famílias de padrões de projeto
- 2. Iterator**  
o que é, como funciona e um exemplo do dia a dia
- 3. Chain of Responsibility**  
o que é, como funciona e um exemplo na programação
- 4. Para que serve cada um**  
a diferença entre os dois, lado a lado

# Onde eles ficam no catálogo

## CRIACIONAIS · 5

Como um objeto nasce.

Factory Method · Abstract Factory · Builder ·  
Prototype · Singleton

## ESTRUTURAIS · 7

Como objetos se compõem.

Adapter · Bridge · Composite · Decorator ·  
Facade · Flyweight · Proxy

## COMPORTAMENTAIS · 11

Como os objetos conversam e dividem  
responsabilidade.

**Chain of Responsibility** · Command ·  
**Iterator** · Mediator · Memento · Observer ·  
State · Strategy · Template Method · Visitor ·  
Interpreter

São 23 padrões no catálogo clássico, de 1994. Os dois de hoje estão na família de comportamento.

# O que é

---

O **Iterator** é um padrão usado para percorrer os elementos de uma coleção um por um, sem precisar saber como essa coleção está armazenada internamente.

Não importa se os dados estão em uma lista, em um vetor ou em outra estrutura: quem percorre não precisa saber.

## Exemplo simples

Uma lista de alunos:



O Iterator entrega: João → Maria → Pedro → Ana

# Como funciona



## As duas operações

**hasNext()** — verifica se existe outro elemento.

**next()** — pega o próximo elemento.

Com essas duas, quem usa a coleção consegue caminhar por ela inteira sem abrir a estrutura por dentro.

# No dia a dia

---

## Uma playlist de músicas

Você não precisa saber como o aplicativo guarda as músicas. Apenas aperta “**Próxima**” e ele passa para a seguinte.

O botão é o `next()`. O que está por trás — lista, fila de download, streaming — é problema do aplicativo, não seu.

## Vantagem

Separa a forma de **percorrer** os dados da forma como os dados são **armazenados**.

Se amanhã a coleção mudar de estrutura, quem percorre continua igual.

# O que é

O **Chain of Responsibility** é um padrão onde uma solicitação passa por uma cadeia de objetos, até que algum deles consiga tratá-la.

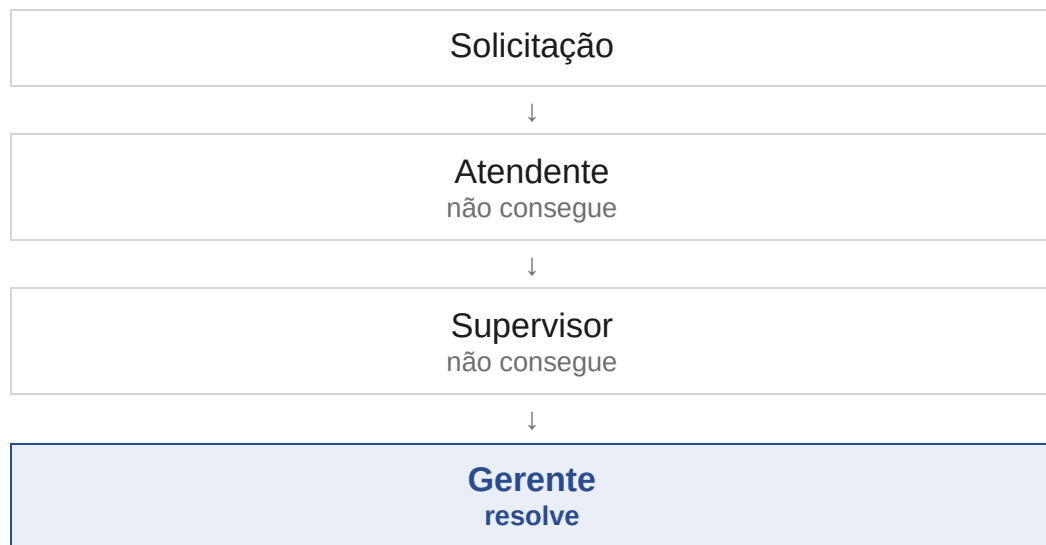
O cliente faz uma solicitação. O atendente tenta resolver. Se não conseguir, passa para o supervisor. Se o supervisor não conseguir, passa para o gerente.

## Exemplo simples: suporte

|            |                   |
|------------|-------------------|
| Cliente    | faz a solicitação |
| Atendente  | tenta resolver    |
| Supervisor | tenta resolver    |
| Gerente    | resolve           |

A solicitação sobe até alguém conseguir tratá-la.

# Como funciona



## Cada participante decide

**“Consigo resolver?”**

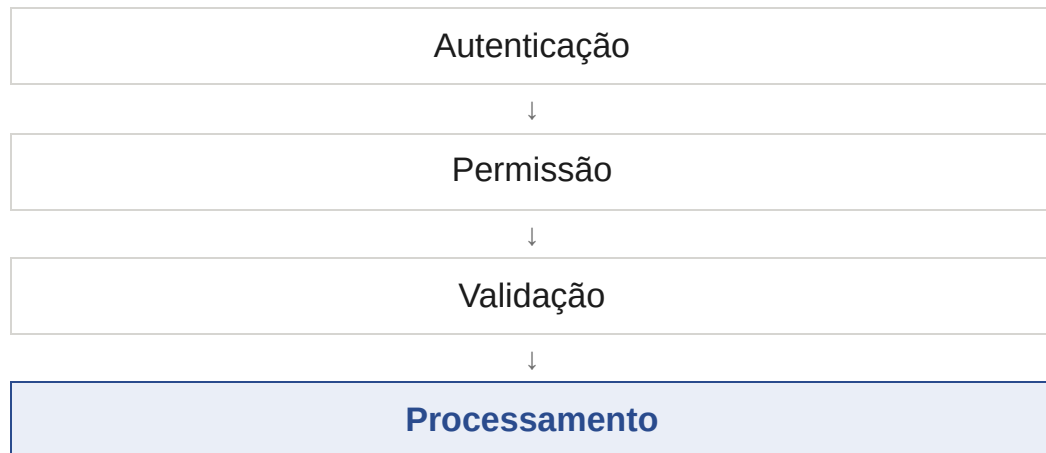
SIM → resolve

NÃO → passa para o próximo

É a única pergunta que cada um precisa responder. Ninguém precisa conhecer a cadeia inteira — só quem vem depois dele.

## Exemplo na programação

Um sistema analisando uma requisição HTTP:



Cada etapa verifica a sua responsabilidade e pode passar a solicitação para a próxima.

### Vantagem

Evita colocar várias regras dentro de uma única classe gigante, distribuindo o tratamento entre diferentes objetos.

Uma regra nova entra como mais uma etapa na cadeia, na posição em que ela vale — sem mexer nas outras.

# Para que serve cada um

| Iterator                    | Chain of Responsibility           |
|-----------------------------|-----------------------------------|
| Percorre elementos          | Processa solicitações             |
| Vai de elemento em elemento | Vai de responsável em responsável |
| Exemplo: lista de alunos    | Exemplo: suporte                  |
| Foco em <b>percorrer</b>    | Foco em <b>processar</b>          |

#### ITERATOR

“Quero passar por todos os elementos, um de cada vez.”

#### CHAIN OF RESPONSIBILITY

“Vou passar essa solicitação para quem conseguir resolver.”

# Obrigado!

Apresentação sobre os padrões Iterator e Chain of Responsibility.

**Arthur Amaral**

**Arthur Gaspare**

**Felipe Ferreira**